Universidad Continental

Trabajo en equipo - Historial de transacciones

Nombres completos:

- Ramìrez Quillatupa Juan Diego
- Garcìa Betancourt Israel Garcia
- Veliz Durand Vieri Del Piero
- Rojas Rivas Eduardo Sebastian
- Leon Armas Luis Aram
- Limas Chagua Jair Roberto
- Benites Fierro Alexander Aiken

Equipo / Grupo: Los Backyardigans

Modulo asignado:

- Historial de transacciones

Roles del equipo:

- DBA : Vieri Veliz Durand
- Dev Frontend: Juan Ramirez
- Dev Backend: Israel Garcìa
- QA Lead: Luis Leon
- Dev UX/UI : Jair Limas , Eduardo Rojas
- DevOps: Alexander Benites

1. **Preguntas:**

1.1. ¿Qué actividades realizaron específicamente en sus roles durante esta tarea?

Como grupo, Vieri verificó la base de datos y controló que no se repitan registros. Luis y Juan se encargaron de separar correctamente el HTML del JS y ordenar la parte visual con Bootstrap. Israel y Alexander revisaron la lógica, integración y estructura del proyecto para que el módulo funcione correctamente. Jair y Eduardo propusieron mejoras visuales y de experiencia de usuario, y todos participaron en la validación y pruebas del módulo.

1.2. ¿Qué fue lo más desafiante y cómo lo resolvieron?

Lo más desafiante fue evitar duplicidad de datos, separar bien el JS sin romper rutas y encontrar mejoras visuales que realmente funcionen. Lo resolvimos revisando el código en conjunto, corrigiendo rutas y estructura de carpetas, y probando varias veces hasta dejar el módulo estable.

1.3. ¿Qué aprendieron que no sabían antes de esta tarea?

Aprendimos a organizarnos mejor por roles, a estructurar el proyecto separando HTML, JS y base de datos, y a usar Bootstrap de manera más ordenada para lograr responsividad. También reforzamos la importancia de revisar la integración completa y no solo partes aisladas.

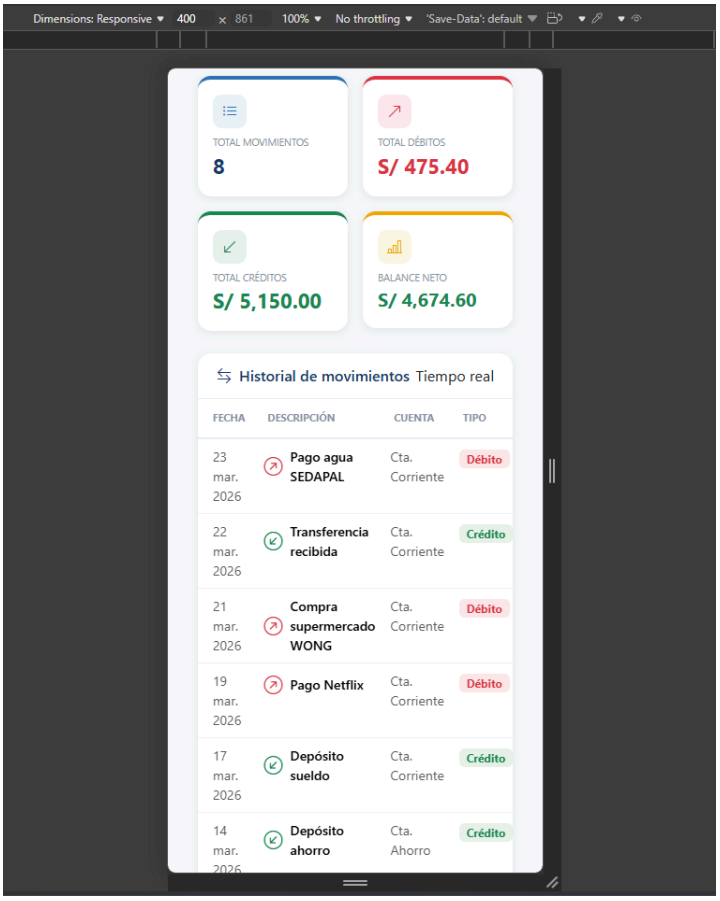
1.4. ¿Cómo evaluarían su propio desempeño? (del 1 al 20) - Justifica su nota

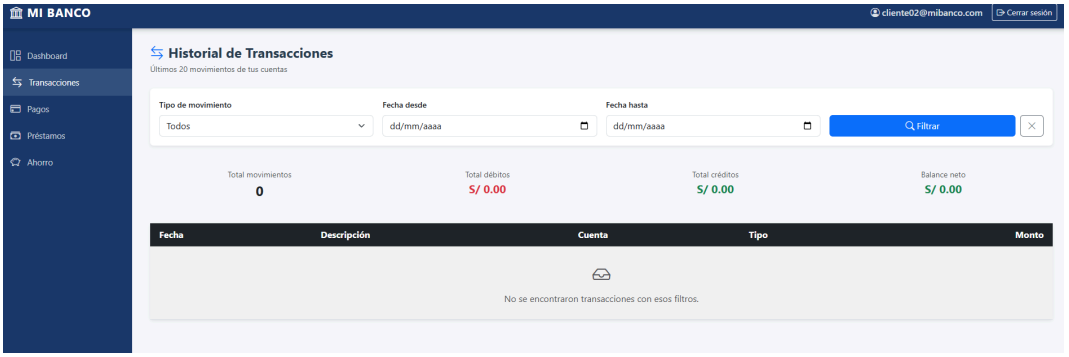
Como grupo nos pondríamos 18/20, porque cumplimos con las tareas principales, hubo aporte de todos los roles y se logró avanzar funcional y visualmente en el módulo. Nos faltó un poco más de orden y rapidez en la integración final, por eso no sería nota máxima.

1.5. ¿Qué mejorarían del trabajo del equipo en la siguiente iteración?

Mejoraríamos la coordinación desde el inicio, definiendo mejor la estructura de carpetas, rutas y revisiones intermedias por rol. También repartiríamos antes las validaciones finales para evitar correcciones apuradas al final.

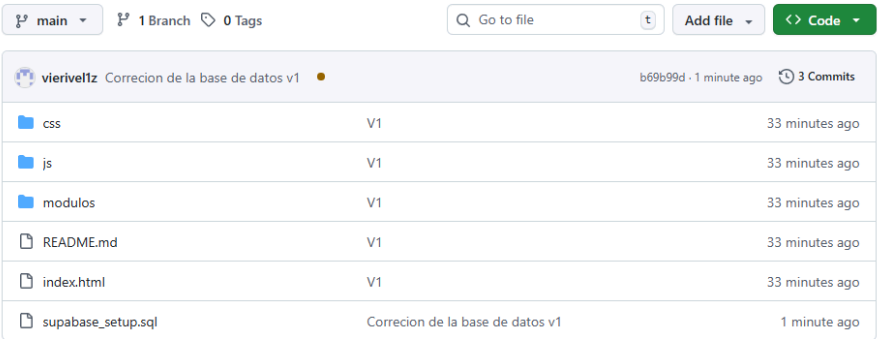
2. Captura del módulo vista en vista mòvil y desktop





3. R1 Administrador de base de datos

a. Script SQL final con correcciones subido al repositorio del equipo.



b. Diccionario de datos en tabla dentro del reporte Word del equipo.

Cuentas		
Campo	**Tipo**	**Descripción**
id	UUID	Identificador único de la cuenta (Clave Primaria).
user_id	UUID	Referencia al usuario dueño de la cuenta (Relación con auth.users).
tipo	TEXT	Categoría de cuenta: solo permite 'corriente' o 'ahorro'.
numero_cuenta	TEXT	El número identificador público/bancario de la cuenta.
saldo	NUMERIC	Cantidad de dinero disponible (por defecto 0).
moneda	TEXT	Tipo de divisa (por defecto 'PEN' - Soles Peruanos).
created_at	TIMESTAMPTZ	Fecha y hora de creación del registro.

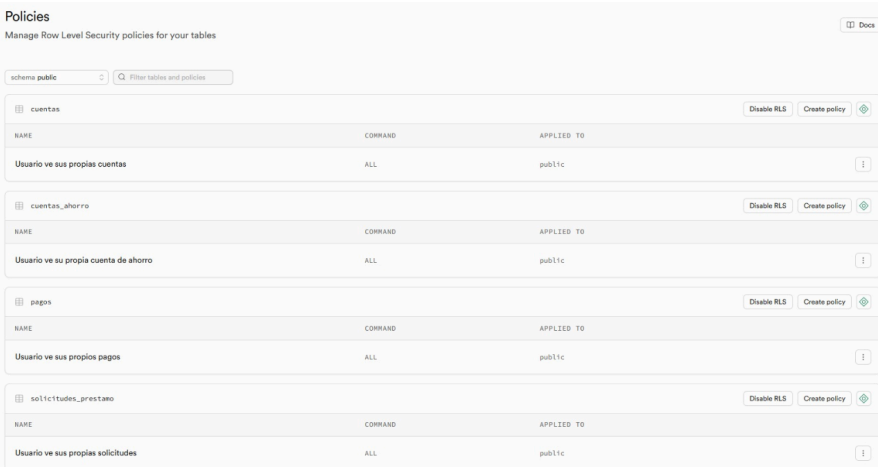
Transacciones		
Campo	**Tipo**	**Descripción**
id	UUID	Identificador único del movimiento.
user_id	UUID	Usuario que realiza o recibe la transacción.
cuenta_id	UUID	Cuenta específica afectada (puede ser nulo si la cuenta se elimina).
tipo	TEXT	Dirección del dinero: 'debito' (salida) o 'credito' (entrada).
descripcion	TEXT	Detalle o motivo del movimiento (ej. "Pago de cena").
monto	NUMERIC	Valor monetario de la transacción.
fecha	TIMESTAMPTZ	Momento exacto en que se realizó la operación.

Pagos		
Campo	**Tipo**	**Descripción**
id	UUID	Identificador único del pago.
user_id	UUID	Usuario que efectuó el pago.
servicio	TEXT	Tipo de servicio: agua, luz, cable, telefono o gas.
numero_contrato	TEXT	Código de cliente o suministro del servicio.
monto	NUMERIC	Total pagado por el servicio.
estado	TEXT	Situación del pago (por defecto 'completado').
fecha	TIMESTAMPTZ	Fecha y hora del pago.

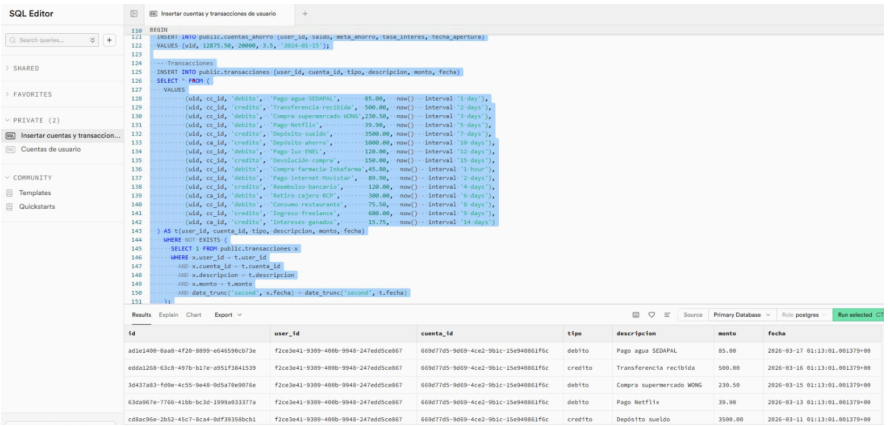
Solicitudes_prestamo		
Campo	Tipo	Descripción
id	UUID	Identificador único de la solicitud.
user_id	UUID	Usuario que solicita el préstamo.
monto	NUMERIC	Capital total solicitado.
plazo_meses	INTEGER	Tiempo de devolución expresado en meses.
tasa_anual	NUMERIC	Porcentaje de interés aplicado al año.
cuota_mensual	NUMERIC	Monto calculado a pagar cada mes.
proposito	TEXT	Justificación del préstamo (opcional).
estado	TEXT	Estado de aprobación (por defecto 'pendiente').
created_at	TIMESTAMPTZ	Fecha de creación de la solicitud.

Cuenta_ahorros		
Campo	Tipo	Descripción
id	UUID	Identificador único de esta cuenta de ahorros.
user_id	UUID	Propietario de la cuenta.
saldo	NUMERIC	Dinero acumulado actualmente.
meta_ahorro	NUMERIC	Objetivo financiero a alcanzar (por defecto 10,000).
tasa_interes	NUMERIC	Porcentaje de ganancia anual (por defecto 3.5%).
fecha_apertura	DATE	Fecha en la que se abrió la cuenta (sin hora).

c. **Captura de las politicas RLS activas en el panel de Supabase**



d. **Captura del Table Editor de Supabase con los datos de prueba insertados.**



Universidad Continental

4. R3 Dev Backend

INSTRUCCIONES

1. Archivo js/modulos/nombreModulo.js con JSDoc en todas las funciones.

transacciones.js

js > modulos > transacciones.js > ...

```javascript
import { supabase, requireAuth, formatSoles, formatFecha } from '../supabase.js';

const user = await requireAuth();
document.getElementById('userName').textContent = user.user_metadata?.full_name?.split(' ')[0] || user.email;

document.getElementById('btnLogout').addEventListener('click', async () => {
    await supabase.auth.signOut();
    window.location.replace('/index.html');
});
```

```javascript
/**
 * Carga las transacciones del usuario desde Supabase aplicando los filtros de fecha y tipo seleccionados en el DOM.
 * Valida que la fecha de inicio no sea mayor a la de fin, muestra un indicador de carga durante la consulta,
 * y luego actualiza la tabla y el resumen en la interfaz.
 * * @async
 * @function cargarTransacciones
 * @returns {Promise<void>} No retorna ningún valor, actualiza el DOM directamente.
 */
async function cargarTransacciones() {
    const desde  = document.getElementById('filtroDesde').value;
    const hasta  = document.getElementById('filtroHasta').value;

    // NUEVA VALIDACIÓN DE FECHAS //
    if (desde && hasta && desde > hasta) {
        alert("La fecha 'Desde' no puede ser mayor que la fecha 'Hasta'.");
        return; // Detenemos la ejecución aquí para no hacer la consulta a Supabase
    }

    mostrarCargando(); /* LLamas a la vista de load NUEVO */

    const tipo   = document.getElementById('filtroTipo').value;

    let query = supabase
        .from('transacciones')
        .select(`*, cuentas(tipo, numero_cuenta)`)
        .eq('user_id', user.id)
        .order('fecha', { ascending: false })
        .limit(20);

    if (tipo)  query = query.eq('tipo', tipo);
    if (desde) query = query.gte('fecha', desde);
    if (hasta) query = query.lte('fecha', hasta + 'T23:59:59');

    const { data: txns, error } = await query;
    renderTabla(txns || []);
    renderResumen(txns || []);
}
```

```javascript
/**
 * Reemplaza el contenido de la tabla de transacciones con un indicador visual de carga (spinner).
 * Se utiliza para dar retroalimentación al usuario mientras se esperan los datos de la base de datos.
 * * @function mostrarCargando
 * @returns {void}
 */
function mostrarCargando() {
  const tbody = document.getElementById('txnBody');
  tbody.innerHTML = `
    <tr>
      <td colspan="5" class="text-center py-4">
        <div class="spinner-border text-primary"></div>
        <p class="text-muted mt-2 mb-0">Buscando transacciones...</p>
      </td>
    </tr>
  `;
}
```

Universidad Continental

```javascript
/**
 * Genera el HTML necesario para mostrar las transacciones y lo inserta en el cuerpo de la tabla.
 * Si el arreglo está vacío, muestra un mensaje indicando que no hay resultados.
 * * @function renderTabla
 * @param {Array<Object>} txns - Arreglo de transacciones a renderizar.
 * @param {string} txns[].fecha - Fecha de la transacción.
 * @param {string} txns[].tipo - Tipo de movimiento ('debito' o 'credito').
 * @param {string} txns[].descripcion - Descripción o concepto de la transacción.
 * @param {number|string} txns[].monto - Importe de la transacción.
 * @param {Object} [txns[].cuentas] - Objeto anidado con los datos de la cuenta vinculada.
 * @param {string} [txns[].cuentas.tipo] - Tipo de cuenta ('corriente' o 'ahorro').
 * @returns {void}
 */
function renderTabla(txns) {
    const tbody = document.getElementById('txnBody');
    if (txns.length === 0) {
        tbody.innerHTML = `<tr><td colspan="5" class="text-center text-muted py-4">
        <i class="bi bi-inbox fs-2 d-block mb-2"></i>No se encontraron transacciones con esos filtros.
        </td></tr>`;
        return;
    }
    tbody.innerHTML = txns.map(t => `
        <tr>
        <td class="ps-3 text-muted small">${formatFecha(t.fecha)}</td>
        <td>
            <div class="d-flex align-items-center gap-2">
            <i class="bi ${t.tipo === 'debito' ? 'bi-arrow-up-right-circle text-danger' : 'bi-arrow-down-left-circle text-success'} fs-5"></i>
            <span class="fw-semibold small">${t.descripcion}</span>
            </div>
        </td>
        <td class="text-muted small">
            ${t.cuentas ? (t.cuentas.tipo === 'corriente' ? 'Cta. Corriente' : 'Cta. Ahorro') : '—'}
        </td>
        <td>
            <span class="badge ${t.tipo === 'debito' ? 'badge-debito' : 'badge-credito'} px-2 py-1">
            ${t.tipo === 'debito' ? 'Débito' : 'Crédito'}
            </span>
        </td>
        <td class="text-end pe-3">
            <span class="${t.tipo === 'debito' ? 'monto-debito' : 'monto-credito'} fw-bold">
            ${t.tipo === 'debito' ? '- ' : '+ '}${formatSoles(t.monto)}
            </span>
        </td>
        </tr>
    `).join('');
}
```

```javascript
/**
 * Calcula los totales de débitos, créditos y el balance neto a partir de las transacciones dadas.
 * Luego, actualiza los elementos correspondientes en el DOM para mostrar el resumen financiero.
 * * @function renderResumen
 * @param {Array<Object>} txns - Arreglo de transacciones para calcular el resumen.
 * @param {string} txns[].tipo - Tipo de movimiento ('debito' o 'credito').
 * @param {number|string} txns[].monto - Importe de la transacción utilizado para las sumas.
 * @returns {void}
 */
function renderResumen(txns) {
    const debitos  = txns.filter(t => t.tipo === 'debito').reduce((s, t)  => s + Number(t.monto), 0);
    const creditos = txns.filter(t => t.tipo === 'credito').reduce((s, t) => s + Number(t.monto), 0);
    const neto = creditos - debitos;

    document.getElementById('resTotal').textContent    = txns.length;
    document.getElementById('resDebitos').textContent  = formatSoles(debitos);
    document.getElementById('resCreditoss').textContent = formatSoles(creditos);

    const netoEl = document.getElementById('resNeto');
    netoEl.textContent = formatSoles(Math.abs(neto));
    netoEl.className   = `fw-bold fs-5 ${neto >= 0 ? 'text-success' : 'text-danger'}`;
}

// Eventos filtros
document.getElementById('btnFiltrar').addEventListener('click', cargarTransacciones);
document.getElementById('btnLimpiar').addEventListener('click', () => {
    document.getElementById('filtroTipo').value  = '';
    document.getElementById('filtroDesde').value = '';
    document.getElementById('filtroHasta').value = '';
    cargarTransacciones();
});

// Carga inicial
cargarTransacciones();
```

Función	Propósito	Líneas de Código	Conexión a Supabase
cargarTransacciones	Carga las transacciones del usuario aplicando filtros de fecha y tipo. Valida las fechas y coordina la actualización de la interfaz llamando a otras funciones.	28	**Sí** (Realiza la consulta `supabase.from('transacciones')...`)
mostrarCargando	Muestra un indicador visual de carga (spinner) en la tabla de HTML mientras se esperan los datos de la base de datos.	11	**No** (Solo manipula el DOM)
renderTabla	Genera e inserta el código HTML dinámico en el DOM para listar las transacciones obtenidas, o muestra un mensaje si no hay resultados.	31	**No** (Solo formatea y manipula el DOM)
renderResumen	Calcula los montos totales de débitos, créditos y el balance neto para actualizar las tarjetas de resumen financiero en la interfaz.	13	**No** (Solo procesa datos y manipula el DOM)

5. **Reporte de funciones**

REPORTE DE FUNCIONES			
Función	**Propósito**	**Líneas**	**Supabase**
initApp	Detecta módulo	6	No
initDashboard	Cargar cuentas	15	Si
renderCuentas	Mostrar cuentas	15	No
initAhorro	Obtener ahorro	15	Si
calcularYRenderAhorro	Lógica + UI	15	No
initPagos	Inicializar pagos	10	No
handlePago	Registrar pago	20	Si
initPrestamos	Inicializar préstamo	10	No
handlePrestamo	Registrar préstamo	25	Si
calcularCuota	Fórmula financiera	10	No

6. **Link del github:** https://github.com/IsraelGarciaBetancourt/SPRINT-2-Sem02.git
7. **Link del Vercel:** sprint-2-sem02-jaox.vercel.app